

Roll No: 2303A52385

Question 1: Frog Jump

Problem description

A frog moves through indexed stones with given heights. It may jump one or two positions and pays the absolute height difference. Find minimum total energy.

Input format

n followed by n heights.

Output format

Print the minimum energy.

Constraints

$2 \leq n \leq 100000$; $0 \leq \text{height}[i] \leq 10000$

Sample input

4
10 20 30 10

Sample output

20

Explanation

Jump 0→1 costs 10 and 1→3 costs 10.

Test Cases

Test	Input	Expected output
1	2 100 1	99
2	6 30 10 60 10 60 50	40

```
class Solution {
    int minCost(int[] height) {
        // code here
        int n=height.length;
        int[] dp=new int[n];
        dp[0]=0;
        for(int i=1;i<n;i++){
            int onestep=dp[i-1]+Math.abs(height[i]-height[i-1]);
            int twostep=Integer.MAX_VALUE;
            if(i>1){
                twostep=dp[i-2]+Math.abs(height[i]-height[i-2]);
            }
            dp[i]=Math.min(onestep,twostep);
        }
        return dp[n-1];
    }
}
```

Question 2: Frog Jump with K Distances

Problem description

The frog may jump from a stone to any of the next k stones. Each jump costs the absolute height difference. Minimize total energy.

Input format

n k , then n heights.

Output format

Print minimum energy.

Constraints

$2 \leq n \leq 100000$

$1 \leq k < n$

Sample input

5 3

10 30 40 50 20

Sample output

10

Explanation

The frog can jump directly from height 10 to height 20 within three positions.

Test Cases:

Test	Input	Expected output
1	5 1 1 100 1 100 1	396
2	6 5 100 1 99 2 98 3	97

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        int n=5;
        int[] arr={10, 30, 40, 50, 20};
        int k=3;
        int[] dp=new int[n];
        Arrays.fill(dp,Integer.MAX_VALUE);
        dp[0]=0;
        for(int i=1;i<n;i++){
            for(int j=1;j<=k;j++){
                if(i-j>=0){
                    int cost=dp[i-j]+Math.abs(arr[i]-arr[i-j]);
                    dp[i]=Math.min(dp[i],cost);
                }
            }
        }
        System.out.print(dp[n-1]);
    }
}
```